



ರೈ ತಾಂತ್ರಿಕ ವಿಶ್ವವಿದ್ಯಾಲಯ
RAI TECHNOLOGY UNIVERSITY

Estd. under Karnataka Act. No. 40 of 2013

Assignment No: 02

Subject Title: Deep Learning

Subject Code: 0CS351T

Submitted By:

Student Name: R Pall Pavan

University Enrolment Number: RTU24101IT027

Program Name: B. TECH IT AIML

Semester: Vth Sem

Session: 2026-2027

Submitted To:

Faculty Name: Mr Kumar Satyam Tanti

College Of Engineering & Computer Application

Rai Technology University

MNIST DIGIT CLASSIFICATION USING NEURAL NETWORK

Aim - To Build a simple Network using TensorFlow/
Keras for classifying handwritten digits
from 0 to 9 using for the MNIST dataset,
evaluate its accuracy, visualize training performa-
nce, test handwritten image, and compare the
results after changing a model parameters.

Introduction: MNIST [Modified National Institute
of Standards and Technology] is a
popular dataset used for handwritten digit
classification. It contains 60,000 training
image is a grayscale image of size
 28×28 pixels and represents one of
the digits from 0 to 9.

In this assignment, a simple neural network is created
using TensorFlow / Keras. The Images are normalised.

and flattened before is given to the network.

The model contains a hidden layer with ReLU activation and an o/p layer with softmax activation.

After training, the model is evaluated using test accuracy.

Objectives:

- Load and explore the MNIST dataset.
- Display sample handwritten digit images.
- Preprocess the image data.
- Build & train a neural network using TensorFlow/Keras.
- Evaluate the model using test accuracy.
- Visualize training & validation accuracy/loss.
- Predict new handwritten images.
- Perform an experiment by changing the number of neurons.

Data Description:

<u>Property</u>	<u>Description</u>
Dataset	MNIST
Training Images	60,000
Testing images	10,000

Image size 28×28

Classes

10

Image type Grayscale.

* The pixel values range from 0 to 255. They are normalized to 0-1 by dividing them by 255.

Methodology:

Load Dataset $\rightarrow$ Explore Data $\rightarrow$ Preprocess $\rightarrow$ Build model
 $\rightarrow$ Train $\rightarrow$ Evaluate $\rightarrow$ Visualize $\rightarrow$ Predict $\rightarrow$ Experiment.

The 28×28 images are flattened into 784 input values. The neural network consists of a hidden layer with 128 neurons using ReLU activation and an o/p layer with 10 neurons using Softmax.

Implementation:

```
import tensorflow as tf
```

```
import matplotlib.pyplot as plt
```

```
import numpy as np
```

```
# Load and Explore MNIST Dataset
```

```
(x_train, y_train), (x_test, y_test) = tf.keras.
```

```
datasets.mnist.load_data()
```

```
Print("Training:", x_train.shape)
```

```
Print("Testing:", x_test.shape)
```

```
plt.imshow(x_train[0], cmap="gray")
```

```
plt.title("Label: {y_train[0]}")
```

```
plt.axis("off")
```

```
plt.show()
```

```
# Preprocessing
```

```
x_train = x_train / 255.0
```

```
x_test = x_test / 255.0
```

```
# Build and Train Neural Network
```

```
model = tf.keras.Sequential([
```

```
    tf.keras.layers.Flatten(input_shape=(28, 28)),
```

```
    tf.keras.layers.Dense(128, activation="relu"),
```

```
    tf.keras.layers.Dense(10, activation="softmax")
```

```
])
```

```
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",  
              metrics=["accuracy"])
```

```
history = model.fit(x_train, y_train, epochs=10, validation-  
                    split=0.2)
```

Model Architecture :

Input Image

28x28



Flatten



Dense layer 128 Neurons RELU



O/P Layer 10 Neurons softmax



Output 0-9

Model Evaluation

loss, accuracy = model.evaluate(x_test, y_test)

print("Test Accuracy:", accuracy)

Visualization of Accuracy and Loss

plt.plot(history.history["accuracy"], label="Training")

plt.plot(history.history["val_loss"], label="validation")

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.legend()

plt.show()

Observation: The training accuracy generally increases with each epoch, while the loss decreases. The validation curves help determine how well the model performs on data that was not directly used for training.

Prediction

```
Pred = np.argmax(model.predict(x_test[:5]), axis=1)
```

```
for i in range(5):
```

```
    print(f"Image {i+1}: Actual = {y_test[i]},
```

```
        Predicted = {pred[i]}")
```

Modified Model:

```
model2 = tf.keras.Sequential([
```

```
    tf.keras.layers.Flatten(input_shape=(28, 28)),
```

```
    tf.keras.layers.Dense(256, activation="relu"),
```

```
    tf.keras.layers.Dense(10, activation="softmax")])
```

```
model2.compile(optimizer="adam",
```

```
               loss="sparse_categorical_crossentropy", metrics=["accuracy"])
```

```
model2.fit(x_train, y_train, epochs=10, validation_split=0.2)
```

```
loss2, acc2 = model2.evaluate(x_test, y_test)
```

Print("Modified Accuracy:", acc2)

Original model

Hidden neurons = 128

Epochs = 10

Modified model

Hidden neurons = 256

Epochs = 10

Observation: Changing the no. of neurons changes the learning capacity of the neural network. The modified model can learn more features because it had more neurons, although the improvement in test accuracy may be small. The exact result depends on training conditions.

Conclusion:

A simple network was successfully implemented using TensorFlow/Keras to classify handwritten digits on the MNIST dataset. The images were normalised & passed through a neural network containing a hidden layer, with ReLU activation & an O/P layer, with softmax activation.

The model achieved high test accuracy and was able to correctly predict handwritten digits.

Training & Validation accuracy & loss were visualized to understand the learning process. Few test images were also used to compare actual & predicted labels.

Finally, the no. of hidden neurons was changed from 128 to 256 to study its effect on model performance. This experiment demonstrated that changing the neural network architecture can affect its accuracy & learning capability.

REPORT

The objective of this experiment is to build a simple neural network using TensorFlow and Keras to classify handwritten digits from 0 to 9. The MNIST dataset is a standard dataset used for image classification and contains grayscale images of handwritten digits. It has 60,000 training images and 10,000 testing images, with each image having a size of 28×28 pixels.

Dataset & Preprocessing.

The MNIST dataset was loaded using `tf.keras.datasets.mnist.load_data()`. The dataset contains images and their corresponding digit labels.

The 28×28 images were flattened into 784 input values before being passed to the dense neural network.

Neural Network Architecture:

The architecture consists of:

- Input Size : 28×28 pixels
- Flatten layer : Converts the images into 784 values.
- Hidden layer : 128 neurons with ReLU activation
- O/P Layer : 10 neurons with softmax activation.

* The O/P layer contains 10 classes corresponding to the digits 0 through 9.

Technologies used:

- Python
- TensorFlow
- Keras
- Numpy
- Matplotlib
- Pandas
- Jupyter Notebook.